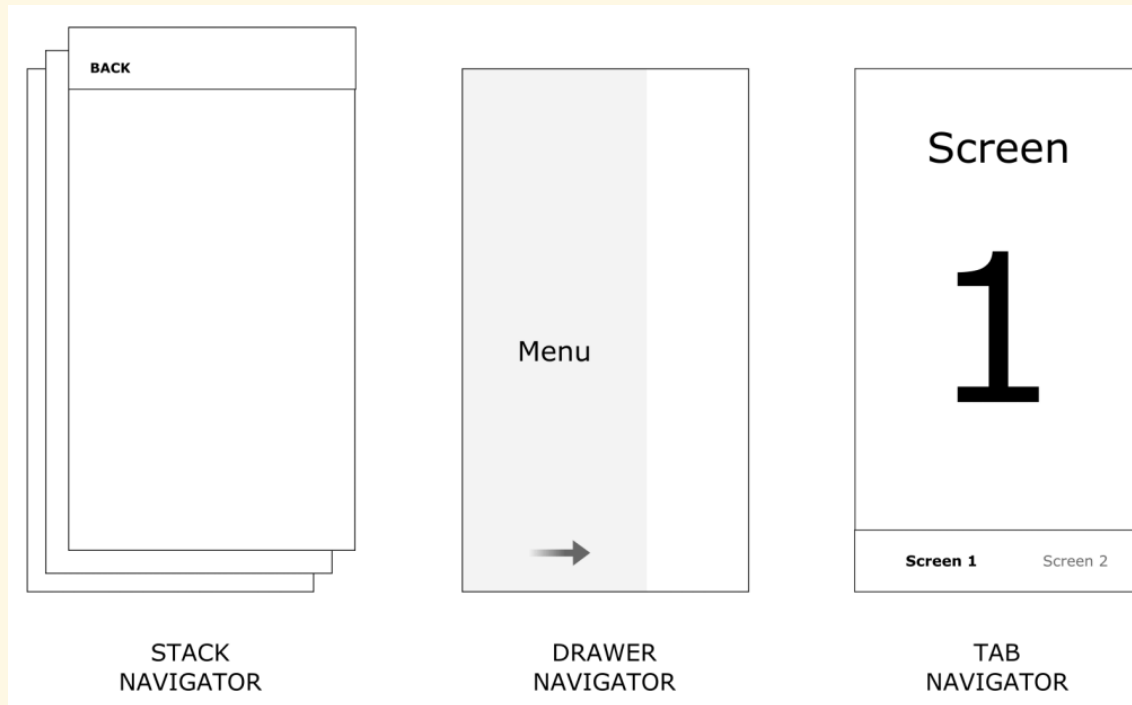

Navigation



Introduction

- ▶ Mobile apps are usually made up of multiple screens
 - ▶ Handling the navigation and controlling the information flow between the screens is important
- ▶ React Navigation
 - ▶ Most popular library for navigation in React Native apps
 - ▶ Common navigation patterns (stack, tab, drawer)



Installation

▶ Common dependencies

▶ Bare projects

```
npm install @react-navigation/native
npm install react-native-screens react-native-safe-area-context
```

▶ Expo projects

```
npm install @react-navigation/native
expo install react-native-screens react-native-safe-area-context
```

▶ Wrap the app in NavigationContainer

```
import { NavigationContainer }
  from "@react-navigation/native";
```

```
export default function App() {
  return <NavigationContainer>
    ...
  </NavigationContainer>
}
```

Stack Navigator

Stack Navigation

- ▶ Transition between screens with navigation history (similar to the navigation in a web browser)

- ▶ Installation

- ▶ `npm install @react-navigation/stack`

- ▶ Usage

- ▶

```
import { createStackNavigator }  
  from "@react-navigation/stack";
```

```
const Stack = createStackNavigator();
```

```
function App() {  
  return <NavigationContainer>  
    <Stack.Navigator>  
      <Stack.Screen name="Screen1" component={Screen1} />  
      <Stack.Screen name="Screen2" component={Screen2} />  
    </Stack.Navigator>  
  </NavigationContainer>  
}
```

Navigating to Another Screen

```
function Screen1({ navigation }) {  
  return <>  
    ...  
    <Button title="Go to Screen 2"  
      onPress={() =>  
        navigation.navigate("Screen2") }  
      />  
    ...  
  </>  
}
```

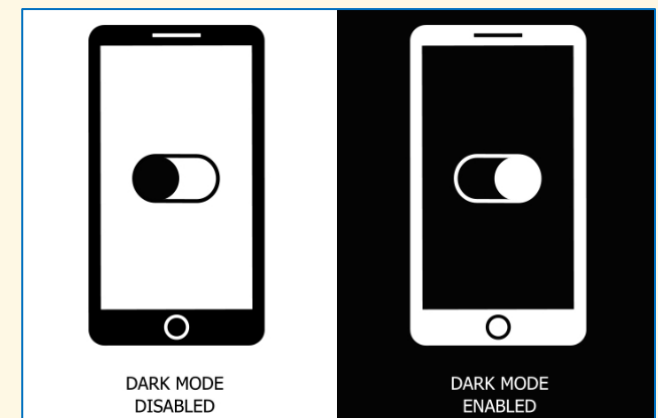
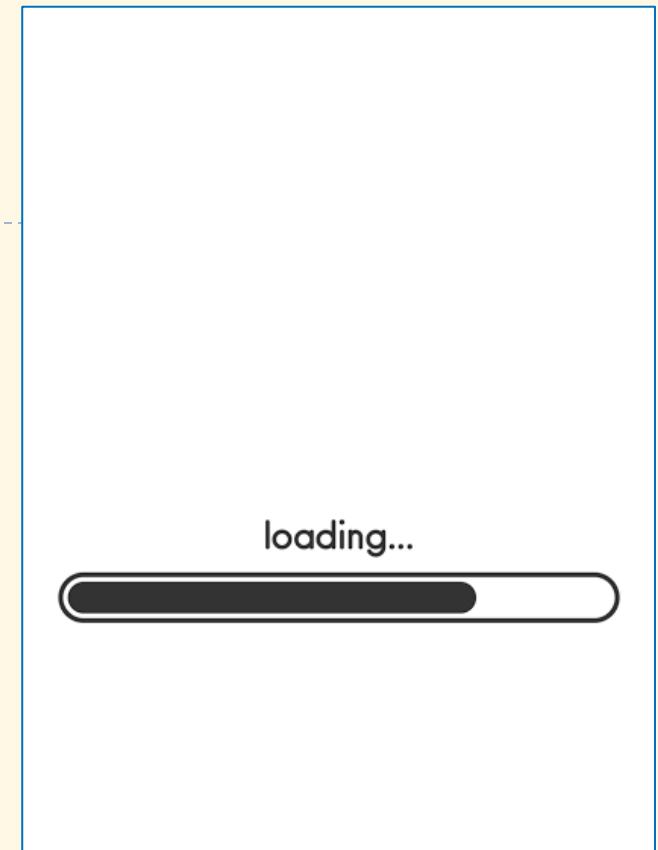
- ▶ Possible to obtain the navigation object with `useNavigation()` hook:
 - ▶ `import {useNavigation}`
from `'@react-navigation/native';`
 - ▶ `const navigation = useNavigation();`

Navigation in History

- ▶ `navigation.goBack()`
 - ▶ Go to the previous screen in navigation history.
- ▶ `navigation.popToTop()`
 - ▶ Go to the first screen in navigation history.
- ▶ `navigation.push("RouteName")`
 - ▶ Add a route on top of the stack and navigates forward to it.
 - ▶ This differs from `navigate()` in that `navigate()` will pop back to earlier in the stack if a component is already mounted there, while `push()` will always add on top, so a component can be mounted multiple times.

Exercises

1. Create an app showing a Welcome screen on startup with a simulated data-loading progress bar, then automatically switch to the Main screen once the data loading completes.
2. Create an app with a main screen called Home and another called Settings that allows to switch the theme between dark and light.



Passing Parameters to Routes on Navigation

- ▶ Useful when the destination screen is customizable

- ▶ Passing parameters:

```
▶ navigation.navigate("CityDetails", {  
    cityId: 10,  
    cityName: "Hanoi",  
    temperature: 26.5  
});
```

- ▶ Reading parameters:

```
▶ function CityDetails({ route, navigation }) {  
    const params = route.params;  
    ...  
}
```

Default Route Parameters

- ▶ Default parameters used when none is given when navigating to a screen

- ▶ Example:

- ▶ `<Stack.Screen`
 `name="CityDetails"`
 `component={CityDetails}`
 `initialParams={{ ... }}`
 `/>`

Header Title

- ▶ Options as an object

- ▶

```
<Stack.Screen
  name="CityDetails"
  component={CityDetails}
  options={{ title: "City details" }}
/>
```

- ▶ Options as a function: useful when the title is dependent on the passed parameters

- ▶

```
<Stack.Screen
  name="CityDetails"
  component={CityDetails}
  options={({route}) => ({ title:
    `City details: ${route.params.cityName}`
  }) }
/>
```

Exercise

- ▶ Create an app with a main screen called Home showing a list, and another screen showing the details of a list item when tapping on it.

Header Style Options

- ▶ `headerStyles`
 - ▶ Style object applied to the `View` that wraps the header
 - ▶ Useful properties: `backgroundColor`
- ▶ `headerTintColor`
 - ▶ Color of the back button and title
- ▶ `headerTitleStyle`
 - ▶ Style object applied to the `Text` that wraps the title
 - ▶ Useful properties: `fontFamily`, `fontWeight`,...

Shared Options Across Screens

```
function StackScreen() {  
    return <Stack.Navigator  
        screenOptions={{  
            // common options  
        }}  
    >  
  
        <Stack.Screen name="Home" component={Home}  
            options={{  
                // specific options  
            }}  
        />  
  
    </Stack.Navigator>  
}
```

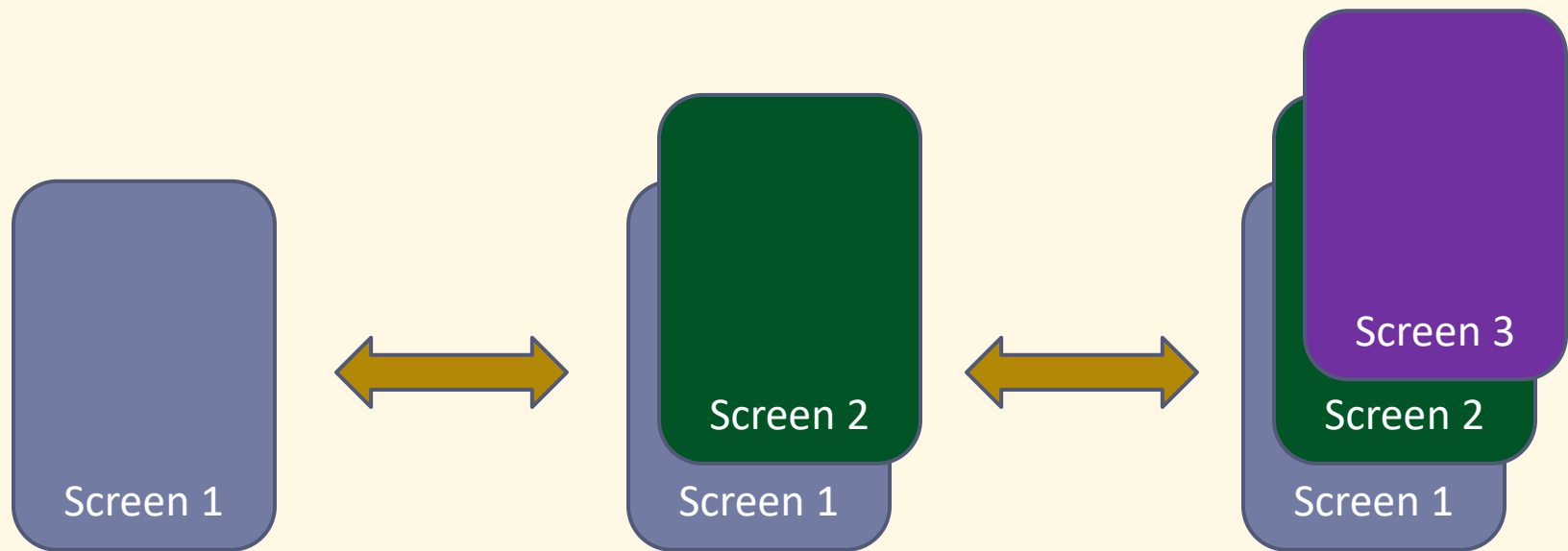
Custom Components on Header

- ▶ `headerTitle`, `headerLeft`, `headerRight` **option** properties can be used to customize the header with given components
- ▶ Example: add a button to the right of the header
 - ▶

```
<Stack.Screen
  name="..."
  component={...}
  options={{
    headerRight: () => <Button ... />
  }}
/>
```

Navigation Lifecycle

- ▶ When navigating to a new screen, the old one remains mounted
- ▶ But when going back to the previous screen, the top one is unmounted



Navigation Events

- ▶ Navigation events:

- ▶ `focus`: when the screen comes into focus
- ▶ `blur`: when the screen goes out of focus

- ▶ Capturing events on screen component:

- ▶

```
function AwesomeScreen({ navigation: nav }) {  
  React.useEffect(() => {  
    const unsubscribe = nav.addListener("focus",  
      () => {  
        // do something  
      });  
    return unsubscribe;  
  }, [navigation]);  
  
  ...  
}
```

- ▶ Attn: `unsubscribe` is used as the clean-up function

Capturing Events on Screen Definition

- ▶ A listener may be added from the component where the navigator is defined rather than inside the screen

- ▶ Example

```
▶ <Stack.Screen
  name="Home"
  component={Home}
  listeners={{
    focus: () => { ... },
    blur: () => { ... }
  }}
/>
```

Drawer Navigator

General

- ▶ Drawer navigator renders a navigation drawer on the side of the screen which can be opened and closed via gestures.

- ▶ Installation

- ▶ `npm install @react-navigation/drawer`
 - ▶ `npm install react-native-gesture-handler react-native-reanimated`

- ▶ Usage

- ▶

```
import {createDrawerNavigator }  
  from "@react-navigation/drawer";
```

```
const Drawer = createDrawerNavigator();
```

```
function App() {  
  return <NavigationContainer>  
    <Drawer.Navigator>  
      <Drawer.Screen name="Screen1" component={Screen1} />  
      <Drawer.Screen name="Screen2" component={Screen2} />  
    </Tab.Navigator>  
  </NavigationContainer>  
}
```

Screen Options

- ▶ Options to be specified under `screenOptions` of navigator or options of screen
 - ▶ `title`: string or function
 - ▶ `drawerLabel`: string or function that given `{focused: boolean, color: string}` returns a `React.Node`
 - ▶ `drawerLabelStyle`: style object for the drawer label
 - ▶ `drawerIcon`: function that given `{focused: boolean, color: string, size: number}` returns a `React.Node`
 - ▶ `drawerIconStyle`: style object for the drawer icon
 - ▶ `drawerStyle`: style object for the drawer component
 - ▶ `drawerPosition`: position of the drawer pane, "left" or "right"
 - ▶ `unmountOnBlur`: whether the screen should be unmounted when navigating away from it
 - ▶ ...

Helper Methods

- ▶ **Open the drawer pane:**
 - ▶ `navigation.openDrawer();`
- ▶ **Close the drawer pane:**
 - ▶ `navigation.closeDrawer();`
- ▶ **Toggle the drawer pane:**
 - ▶ `navigation.toggleDrawer();`

Tab Navigator

Tabbed Navigation

- ▶ A tab bar allowing to switch between routes. Routes are lazily initialized.

- ▶ **Installation**

- ▶ `npm install @react-navigation/bottom-tabs`

- ▶ **Usage**

- ▶

```
import { createBottomTabNavigator }  
  from "@react-navigation/bottom-tabs";
```

```
const Tab = createBottomTabNavigator();
```

```
function App() {  
  return <NavigationContainer>  
    <Tab.Navigator>  
      <Tab.Screen name="Screen1" component={Screen1} />  
      <Tab.Screen name="Screen2" component={Screen2} />  
    </Tab.Navigator>  
  </NavigationContainer>  
}
```


Screen Options

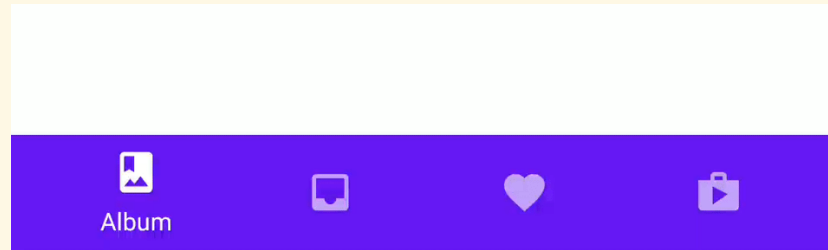
- ▶ Options to be specified under `screenOptions` of navigator or options of screen
 - ▶ `title`: string or function
 - ▶ `tabBarLabel`: string or function that given `{focused: boolean, color: string}` returns a `React.Node`
 - ▶ `tabBarLabelStyle`: style object for the tab label
 - ▶ `tabBarIcon`: function that given `{focused: boolean, color: string, size: number}` returns a `React.Node`
 - ▶ `tabBarIconStyle`: style object for the tab icon
 - ▶ `tabBarStyle`: style object for the tab bar
 - ▶ `unmountOnBlur`: whether the screen should be unmounted when navigating away from it
 - ▶ ...

Navigation Events

- ▶ `focus`
- ▶ `blur`
- ▶ `tabPress`: user presses the tab button for the current screen in the tab bar
- ▶ `tabLongPress`: user presses the tab button for the current screen in the tab bar for an extended period

Material Bottom Tabs

- ▶ A material-design themed tab bar on the bottom of the screen. Routes are lazily initialized.



- ▶ **Installation**

- ▶ `npm install @react-navigation/material-bottom-tabs react-native-vector-icons`

- ▶ **Usage**

- ▶

```
import { createMaterialBottomTabNavigator }  
  from '@react-navigation/material-bottom-tabs';  
import MaterialCommunityIcons  
  from 'react-native-vector-icons/MaterialCommunityIcons';  
  
const Tab = createMaterialBottomTabNavigator();  
...
```

Example

```
function MyTabs() {
  return <Tab.Navigator initialRouteName="Feed" activeColor="#e91e63"
    barStyle={{ backgroundColor: 'tomato' }}>
    <Tab.Screen name="Feed" component={Feed}
      options={{
        tabBarLabel: 'Home',
        tabBarIcon: ({color})
          => <MaterialCommunityIcons name="home" color={color} size={26} />
      }} />
    <Tab.Screen name="Notifications" component={Notifications}
      options={{
        tabBarLabel: 'Updates',
        tabBarIcon: ({color})
          => <MaterialCommunityIcons name="bell" color={color} size={26} />
      }} />
    <Tab.Screen name="Profile" component={Profile}
      options={{
        tabBarLabel: 'Profile',
        tabBarIcon: ({color})
          => <MaterialCommunityIcons name="account" color={color} size={26} />
      }} />
  </Tab.Navigator>
}
```

Material Top Tabs

- ▶ A material-design themed tab bar on the top of the screen. Screen components are mounted immediately.
- ▶ Installation
 - ▶ `npm install @react-navigation/material-top-tabs react-native-tab-view`
- ▶ Usage
 - ▶

```
import { createMaterialTopTabNavigator }
  from '@react-navigation/material-top-tabs';

const Tab = createMaterialTopTabNavigator();

function MyTabs() {
  return <Tab.Navigator>
    <Tab.Screen name="Home" component={HomeScreen} />
    <Tab.Screen name="Settings"
component={SettingsScreen} />
  </Tab.Navigator>
}
```

Nesting Navigators

Introduction

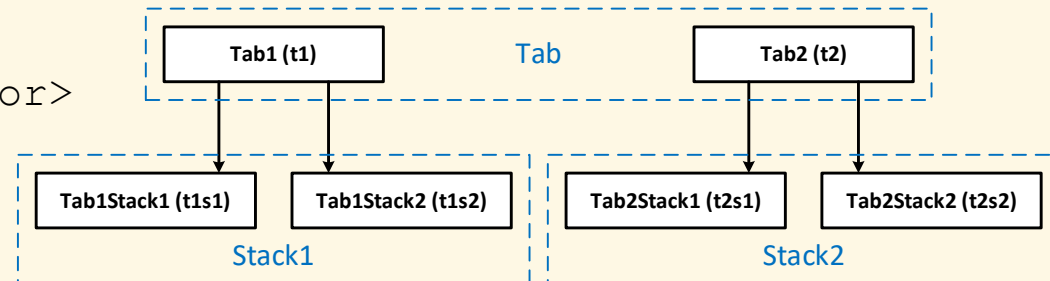
- ▶ Nesting navigators means rendering a navigator inside a screen of another navigator

- ▶ Example:

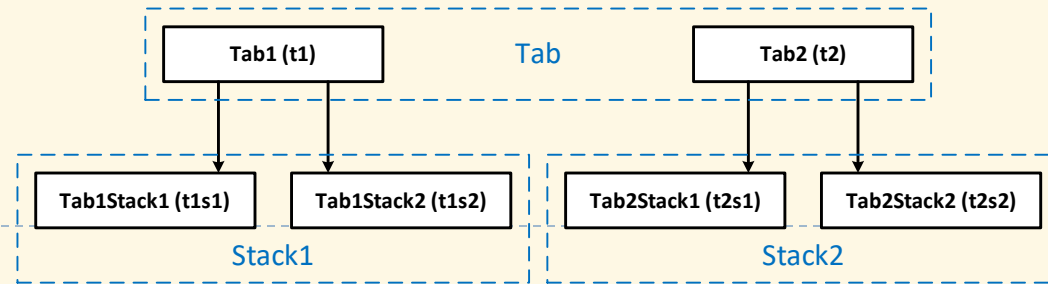
```
function Tab1() {  
  return <Stack1.Navigator>  
    <Stack1.Screen name="t1s1" component={Tab1Stack1} />  
    <Stack1.Screen name="t1s2" component={Tab1Stack2} />  
  </Stack1.Navigator>  
}
```

```
function Tab2() {  
  return <Stack2.Navigator>  
    ...  
  </Stack2.Navigator>  
}
```

```
function Main() {  
  return <NavigationContainer>  
    <Tab.Navigator screenOptions={{ headerShown: false }}>  
      <Tab.Screen name="t1" component={Tab1} />  
      <Tab.Screen name="t2" component={Tab2} />  
    </Tab.Navigator>  
  </NavigationContainer>  
}
```



Navigating



- ▶ **From t1s1 to t1s2:**
 - ▶ `navigation.navigate("t1s2");`
- ▶ **From t1s1 to t2:**
 - ▶ `navigation.navigate("t2");`
 - ▶ Navigation actions are handled by current navigator and bubble up if couldn't be handled
- ▶ **From t1s1 to t2s2:**
 - ▶ `navigation.navigate("t2", {
 screen: "t2s2"
});`
- ▶ **From t1s1 to t2s2 and passing parameters:**
 - ▶ `navigation.navigate("t2", {
 screen: "t2s2",
 params: { ... }
});`

Homework

- ▶ Create a Tap Challenge game with 3 tabbed screens:
 - ▶ Game: allows to play the game.
 - ▶ The user need to tap on a box that appears at a random location on the screen.
 - ▶ Once the box is tapped, it reappears at another location.
 - ▶ The score for one game is the number of times that the user correctly taps the box.
 - ▶ Stats: displays the high score and allows to reset it.
 - ▶ Settings: allows to change the box size and color.

